

Running & Testing Guide — Mobile API

Step-by-step from a fresh clone to a working `curl` against every endpoint. Takes about 15 minutes the first time, ~2 minutes after that.

0. Prerequisites

- **Node.js 20.19+** — `node --version`
 - **npm 10+** — `npm --version`
 - A **Supabase project** — you already have one at `xdtrqkjztgjhtahmbjx.supabase.co`
 - A terminal: `bash`, `zsh`, or `PowerShell` (commands work in all three)
 - `curl` for the smoke tests below (or use Postman/Insomnia/Bruno if you prefer)
-

1. Apply the SQL migration

This creates all the new tables (`api_keys`, `refresh_tokens`, `idempotency_keys`, `repair_updates`, `audit_cycles`, `audit_verifications`, `asset_photos`, `api_key_usage_log`) and adds `users.password_hash`.

1. Open the Supabase Dashboard → **SQL Editor** → **New query**
2. Copy the contents of [supabase/schema v26 mobile api.sql](#) into the editor
3. Click **Run**
4. Verify in **Table Editor** that the new tables appear

⚠ **The migration is idempotent** — re-running it is safe. It uses `CREATE TABLE IF NOT EXISTS` and `ADD COLUMN IF NOT EXISTS` everywhere.

2. Configure the server

```
cd server
cp .env.example .env
```

Open `server/.env` in your editor and fill in **three required values**:

```
SUPABASE_URL=https://xdtrqkjztgjhtahmbjx.supabase.co
SUPABASE_SERVICE_ROLE_KEY=<paste from Supabase → Settings → API →
service_role secret>
```

```
JWT_SECRET=<run: openssl rand -base64 48>
```

If you don't have `openssl`, generate the secret in Node:

```
node -e "console.log(require('crypto').randomBytes(48).toString('base64'))"
```

Optionally tighten CORS to your dev origin:

```
CORS_ORIGINS=http://localhost:5173
```

The other defaults (port 4000, 30 min access token, 30 day refresh, 120 req/min per key) are fine for development.

3. Install + run the server

```
# inside server/  
npm install      # ~15s, you've already done this  
npm run dev
```

You should see:

```
1xL Asset Tracker API listening on http://localhost:4000 (development)
```

Leave this terminal open. The server hot-reloads on file changes (`tsx watch`).

Smoke test — health

In a second terminal:

```
curl http://localhost:4000/health
```

Expected:

```
{  
  "ok": true,  
  "service": "1xl-asset-tracker-api",  
  "env": "development",  
  "uptimeSeconds": 5,  
  "db": { "ok": true, "latencyMs": 87 },  
  "minAppVersion": { "ios": "1.0.0", "android": "1.0.0" },  
  "maintenanceMessage": null  
}
```

`db.ok: true` means the service-role key works and the migration applied. If `db.ok: false`, double-check `SUPABASE_URL` + `SUPABASE_SERVICE_ROLE_KEY`.

4. Configure the web app

In the **repo root** (not `server/`):

```
# Add to .env (next to the existing VITE_SUPABASE_* lines):  
echo "VITE_MOBILE_API_URL=http://localhost:4000" >> .env
```

If you already had a value there, edit it instead of appending.

5. Run the web app

In a third terminal, from the repo root:

```
npm install      # only if you haven't already  
npm run dev
```

Vite serves on `http://localhost:5173`.

6. Create your first API key

1. Open `http://localhost:5173` in a browser
2. Log in **as an admin user** (any user with `role = 'admin'`)
3. In the sidebar: **Administration** → **API Keys**
4. Click **Create key**
 - **Name:** `Local dev - mobile`
 - **Rate limit:** leave at 120/min
 - **Expires:** leave blank
 - **Scopes:** click **Mobile defaults** (selects all the scopes the mobile app needs)
5. Click **Create key**
6. **Copy the secret** that appears — it starts with `1x1_test_...`. You won't see it again.

Save it as a shell variable:

```
export API_KEY="1x1_test_paste_your_key_here"
```

7. Test the API end-to-end with curl

Each section below assumes `$API_KEY` is set from step 6 and that you have at least one **active user** in the org you're testing.

7.1 Login → get a JWT

```
curl -X POST http://localhost:4000/api/v1/auth/login \
  -H "Content-Type: application/json" \
  -H "X-API-Key: $API_KEY" \
  -d '{"email":"admin@example.com","password":"your-password"}'
```

Response (truncated):

```
{
  "accessToken": "eyJhbGciOiJIUzI1NiIs... ",
  "accessTokenExpiresIn": 1800,
  "refreshToken": "rT8K...",
  "refreshTokenExpiresIn": 2592000,
  "user": { "id": "...", "name": "...", "email": "admin@example.com", "role":
"admin", ... }
}
```

Save both tokens:

```
export ACCESS="<paste accessToken>"
export REFRESH="<paste refreshToken>"
```

7.2 Whoami

```
curl http://localhost:4000/api/v1/auth/me \
  -H "X-API-Key: $API_KEY" \
  -H "Authorization: Bearer $ACCESS"
```

Returns { user, organization }.

7.3 List your organization's users (reference data)

```
curl "http://localhost:4000/api/v1/reference/users?limit=10" \
  -H "X-API-Key: $API_KEY" \
  -H "Authorization: Bearer $ACCESS"
```

7.4 Look up an asset by QR

```
curl -X POST http://localhost:4000/api/v1/assets/lookup-by-qr \
  -H "Content-Type: application/json" \
  -H "X-API-Key: $API_KEY" \
  -H "Authorization: Bearer $ACCESS" \
  -d '{"qrPayload":"LAP-1XL-01-001"}'
```

Or pass an asset id (UUID) or even a full QR-URL like

`https://asset.thebizzfly.com/1xl/scan/<assetId>`. All three formats are accepted.

Response is the **asset envelope**: asset + location + department + vendor + current allocation + assignee + audit context (if a cycle is active) + permittedActions (the action tiles mobile should render).

7.5 Raise a repair (with idempotency)

```
ASSET_ID=<paste an asset id from 7.4>
IDEM=$(uuidgen | tr A-Z a-z)

curl -X POST http://localhost:4000/api/v1/assets/$ASSET_ID/repairs \
  -H "Content-Type: application/json" \
  -H "X-API-Key: $API_KEY" \
  -H "Authorization: Bearer $ACCESS" \
  -H "Idempotency-Key: $IDEM" \
  -d '{
    "issue": "Battery does not charge past 80%",
    "priority": "medium",
    "notes": "Reproduced 3 times today"
  }'
```

Run the **same command again** — same `Idempotency-Key`, same body — and you'll get back the cached response (status 201, identical body). Try again with a different body and you'll get 409 CONFLICT.

7.6 Append an update to that repair

```
REPAIR_ID=<from 7.5 response>

curl -X POST http://localhost:4000/api/v1/repairs/$REPAIR_ID/updates \
  -H "Content-Type: application/json" \
  -H "X-API-Key: $API_KEY" \
  -H "Authorization: Bearer $ACCESS" \
  -H "Idempotency-Key: $(uuidgen)" \
  -d '{
    "statusTo": "in_progress",
    "note": "Picked up by IT, diagnosing"
  }'
```

7.7 Log maintenance

```
curl -X POST http://localhost:4000/api/v1/assets/$ASSET_ID/maintenance \
  -H "Content-Type: application/json" \
  -H "X-API-Key: $API_KEY" \
  -H "Authorization: Bearer $ACCESS" \
  -H "Idempotency-Key: $(uuidgen)" \
  -d '{
    "taskId": "cleaning",
    "type": "preventive",
    "notes": "Cleaned vents and screen",
    "checklist": ["Vents cleaned", "Screen wiped"]
  }'
```

7.8 Photo upload (3-step flow)

```
# 1. ask the server for a presigned URL
RESP=$(curl -s -X POST
http://localhost:4000/api/v1/assets/$ASSET_ID/photos/upload-url \
```

```

-H "Content-Type: application/json" \
-H "X-API-Key: $API_KEY" \
-H "Authorization: Bearer $ACCESS" \
-d '{"filename":"shot.jpg","mimeType":"image/jpeg","sizeBytes":12345}'}
echo $RESP | jq

PHOTO_ID=$(echo $RESP | jq -r .photoId)
UPLOAD_URL=$(echo $RESP | jq -r .uploadUrl)

# 2. PUT the actual bytes to Supabase Storage
curl -X PUT "$UPLOAD_URL" \
-H "Content-Type: image/jpeg" \
-H "x-upsert: false" \
--data-binary @./shot.jpg

# 3. tell the server to flip pending → active and append to the asset
curl -X POST
"http://localhost:4000/api/v1/assets/$ASSET_ID/photos/$PHOTO_ID/finalize" \
-H "Content-Type: application/json" \
-H "X-API-Key: $API_KEY" \
-H "Authorization: Bearer $ACCESS" \
-d '{"caption":"Front view after cleaning"}'

```

7.9 Recovery (atomic — optionally marks asset dead)

```

curl -X POST http://localhost:4000/api/v1/assets/$ASSET_ID/recovery \
-H "Content-Type: application/json" \
-H "X-API-Key: $API_KEY" \
-H "Authorization: Bearer $ACCESS" \
-H "Idempotency-Key: $(uuidgen)" \
-d '{
  "incidentType": "stolen",
  "severity": "high",
  "description": "Reported missing from desk 4B",
  "estimatedLoss": 1200,
  "markAssetDead": true
}'

```

If `markAssetDead: true` and the asset-status update fails, the recovery row is rolled back so you never end up half-committed.

7.10 Refresh access token

```

curl -X POST http://localhost:4000/api/v1/auth/refresh \
-H "Content-Type: application/json" \
-H "X-API-Key: $API_KEY" \
-d '{"refreshToken":"$REFRESH"}'

```

Returns a new `accessToken` AND a new `refreshToken` (rotation). Replace your local `$ACCESS` and `$REFRESH` after each refresh.

Reuse a previously-rotated refresh token and **all** of that user's sessions are revoked immediately — that's the reuse-detection defence. You'll get `401 UNAUTHENTICATED` and need to log in again.

7.11 Logout

```
curl -X POST http://localhost:4000/api/v1/auth/logout \
  -H "Content-Type: application/json" \
  -H "X-API-Key: $API_KEY" \
  -d "{\"refreshToken\":\"$REFRESH\",\"allDevices\":false}"
```

Returns `{ "ok": true }` even if the token didn't exist (idempotent).

8. Watch usage in the admin page

Back in the browser at **API Keys**, click **Usage** on your test key. You'll see:

- Total requests in the window
- Success / error counts
- Top endpoints
- Top error codes
- A scrollable list of every request: timestamp, method, endpoint, status, latency, IP

Toggle the window between **1h** / **6h** / **1d** / **7d** / **30d**.

9. Common errors and what they mean

HTTP	error.code	What to do
401	UNAUTHENTICATED	Missing/bad/expired token. Re-login or refresh.
401	TOKEN_EXPIRED	Access token aged out. Call <code>/auth/refresh</code> .
401	INVALID_API_KEY	<code>X-API-Key</code> is missing, malformed, revoked, or expired.

HTTP	<code>error.code</code>	What to do
403	FORBIDDEN	User exists but isn't allowed (e.g. cross-org).
403	INSUFFICIENT_SCOPE	API key doesn't have the scope this endpoint needs. Edit the key.
404	ASSET_NOT_FOUND / REPAIR_NOT_FOUND / PHOTO_NOT_FOUND / CYCLE_NOT_FOUND	Self-explanatory.
400	INVALID_QR_PAYLOAD	The string in <code>qrPayload</code> didn't match a UUID, asset tag, or known URL pattern.
422	VALIDATION_FAILED	Body shape wrong. Check <code>error.fieldErrors</code> .
409	CONFLICT	Idempotency-Key reused with a different body, OR business-rule conflict (e.g. recovery on already-dead asset).
429	RATE_LIMITED	Per-key (120/min) or per-IP login (20 per 15 min). Honour <code>Retry-After</code> .
500	INTERNAL_ERROR	Server-side bug. The <code>requestId</code> in the response will let you grep server logs.

Every error envelope includes `requestId` — log it in mobile crash reports.

10. Troubleshooting

Symptom	Likely cause	Fix
Connection refused from curl	Server not running	<code>cd server && npm run dev</code>
Server exits immediately with "Invalid environment variables"	<code>.env</code> missing required keys	Re-run step 2; check <code>JWT_SECRET</code> <code>>= 32 chars</code>
<code>db.ok: false</code> in <code>/health</code>	Wrong service-role key, or migration not applied	Re-paste service-role from Supabase dashboard; re-run migration
Web "Could not load API keys" toast	<code>VITE_MOBILE_API_URL</code> not set, or server not running	Step 4 + check terminal 1
<code>INVALID_API_KEY</code> on every request	Copy-paste lost a character	Recreate the key, copy more carefully
<code>INSUFFICIENT_SCOPE</code>	Created the key with too few scopes	Edit the key in the admin page → toggle the missing scope
Login works but every other call returns 401	<code>Authorization: Bearer</code> header not sent, or token expired (>30 min)	Send the header; refresh the token
<code>429 RATE_LIMITED</code> during	Hit the per-key bucket or login	Wait 60s (per-key) or 15 min (login), or bump the key's

Symptom	Likely cause	Fix
testing	bucket	<code>rateLimitPerMinute</code>
CORS error in browser	<code>CORS_ORIGINS</code> doesn't include your dev origin	Add <code>http://localhost:5173</code> to <code>CORS_ORIGINS</code> and restart server
<code>EADDRINUSE: 4000</code> on server start	Another process is on port 4000	Set <code>PORT=4001</code> in <code>.env</code> and update <code>VITE_MOBILE_API_URL</code>
Photo <code>finalize</code> returns "Object not found in storage"	Step 7.8 (2) failed silently	Check the <code>PUT</code> returned 200; re-upload to the same <code>uploadUrl</code>
<code>INVALID_QR_PAYLOAD</code> on every QR test	Asset tag has lowercase letters or non-allowed chars	Use uppercase asset tag, or pass the asset's UUID

11. Building for production

```
cd server
npm run build      # outputs dist/
npm start          # runs node dist/index.js
```

Set `NODE_ENV=production` and a real `CORS_ORIGINS=https://asset.thebizzfly.com`. Make sure your reverse proxy forwards `X-Forwarded-For` so per-IP rate limits work.

For a one-line deploy on Render/Fly/Railway:

```
build:  npm ci && npm run build
start:  npm start
healthcheck: /health
```

12. What to give the Flutter team

1. The base URL (production or staging)

2. **One** `X-API-Key` issued from the admin UI (mobile-default scopes)
3. The OpenAPI spec: [server/openapi.yaml](#)
4. The full reference: [server/API.md](#)
5. Their test login credentials (any user account in the test org)

That's the complete handoff packet.